

Documentazione Test

1. Test di Unità

Package: `utility`

Classe Testata: `Keyboard`

Test 1 (`testReadIntConInputNonValido`)

Significato: verifica il comportamento difensivo di `readInt()` in caso di input non numerico, garantendo che l'eccezione `NumberFormatException` venga gestita internamente senza propagarsi al chiamante.

Campo	Contenuto
Input	stringa <code>"ciao"</code> fornita come input
Azione	<code>Keyboard.readInt()</code>
Atteso	restituisce esattamente <code>Integer.MIN_VALUE</code> (<code>-2147483648</code>); il contatore degli errori viene incrementato di <code>1</code>

Test 2 (`testReadDoubleConInputNonValido`)

Significato: verifica il comportamento difensivo di `readDouble()` in caso di input non numerico, assicurando che l'eccezione venga gestita internamente senza interrompere il flusso del programma.

Campo	Contenuto
Input	stringa <code>"ciao"</code> fornita come input
Azione	<code>Keyboard.readDouble()</code>
Atteso	restituisce <code>Double.NaN</code> ; il contatore degli errori viene incrementato

Test 3 (`testReadCharConInputNonValido`)

Significato: verifica il comportamento di `readChar()` in caso di errore di lettura, mantenendo la coerenza con gli altri metodi della classe.

Campo	Contenuto
Input	un input che causa un'eccezione durante la lettura
Azione	<code>Keyboard.readChar()</code>
Atteso	restituisce <code>Character.MIN_VALUE</code> ; il contatore degli errori viene incrementato

Test 4 (`testIncrementoContatoreErroriMultiplo`)

Significato: verifica il corretto accumulo del contatore errori attraverso errori consecutivi, dimostrando che lo stato viene mantenuto tra diverse chiamate.

Campo	Contenuto
Input	due letture errate consecutive: <code>"errore1\n"</code> seguito da <code>"errore2\n"</code>
Azione	<code>Keyboard.readInt()</code> sull'input <code>"errore1\n"</code> , poi <code>Keyboard.readDouble()</code> sull'input <code>"errore2\n"</code>
Atteso	il contatore degli errori viene incrementato correttamente a <code>2</code>

Test 5 (`testReadIntConInputValido`)

Significato: verifica il corretto funzionamento di `readInt()` con input valido, confermando che il comportamento normale non sia influenzato dalla logica di gestione degli errori.

Campo	Contenuto
Input	valore <code>"42\n"</code>
Azione	<code>Keyboard.readInt()</code>
Atteso	restituisce il valore intero <code>42</code> ; il contatore degli errori rimane a <code>0</code>

Test 6 (`testReadWordSingoloToken`)

Significato: verifica la corretta estrazione di un singolo token tramite `readWord()` , basata sull'uso di `StringTokenizer` .

Campo	Contenuto
Input	"Hello World\n"
Azione	Keyboard.readWord()
Atteso	restituisce solo il primo token, "Hello", senza incrementare il contatore degli errori

Test 7 (testReadStringConcatenaToken)

Significato: verifica la concatenazione di tutti i token fino a fine riga tramite `readString()`, preservando gli spazi originali come previsto dall'implementazione.

Campo	Contenuto
Input	"Hello World Test\n"
Azione	Keyboard.readString() (recupera il primo token e itera concatenando tutti i token successivi fino a <code>endOfLine()</code>)
Atteso	restituisce "Hello World Test", con gli spazi originali preservati; il contatore degli errori non viene incrementato

Test 8 (testReadCharConBuffer)

Significato: verifica il meccanismo di buffering in `readChar()`, che salva i caratteri rimanenti in `current_token` per le letture successive.

Campo	Contenuto
Input	"ab\nc\n"
Azione	tre chiamate successive a <code>Keyboard.readChar()</code>
Atteso	la prima lettura estrae 'a' e salva "b" nel buffer; la seconda legge 'b' senza consumare nuovo input; la terza consuma la riga successiva leggendo 'c'; il contatore degli errori resta a 0 in tutte e tre le letture